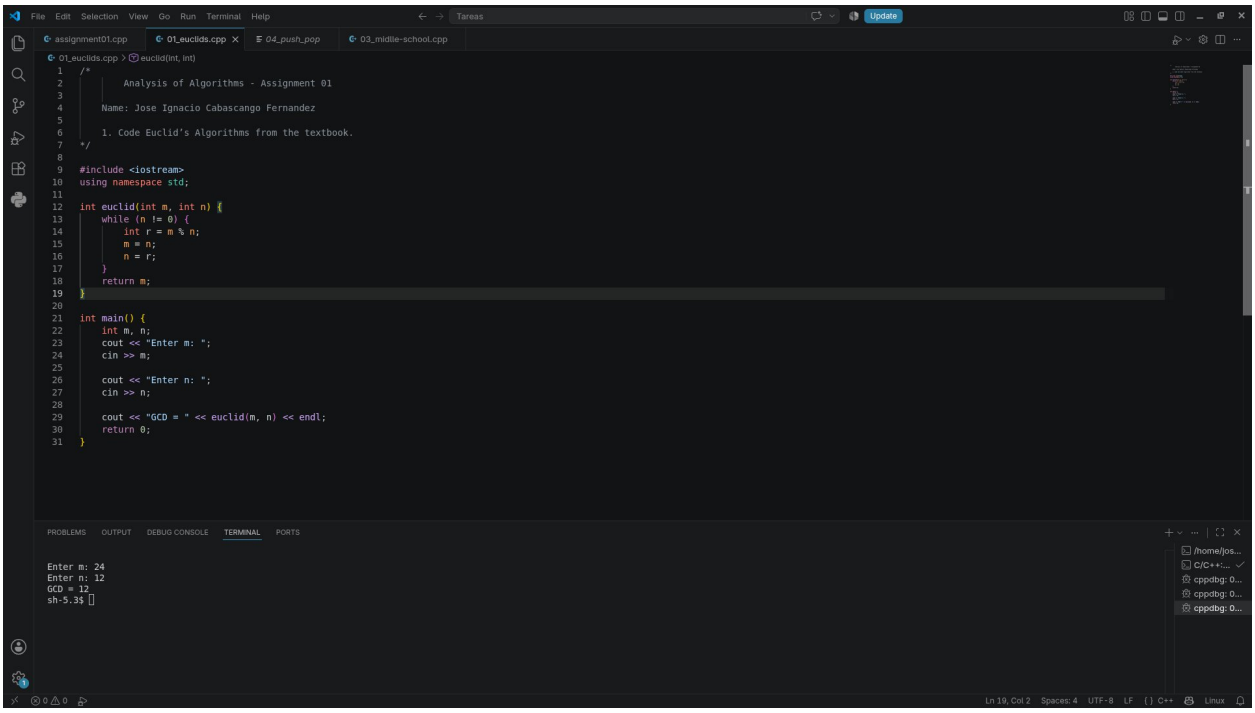
Analysis of Algoritms

Assignament 01

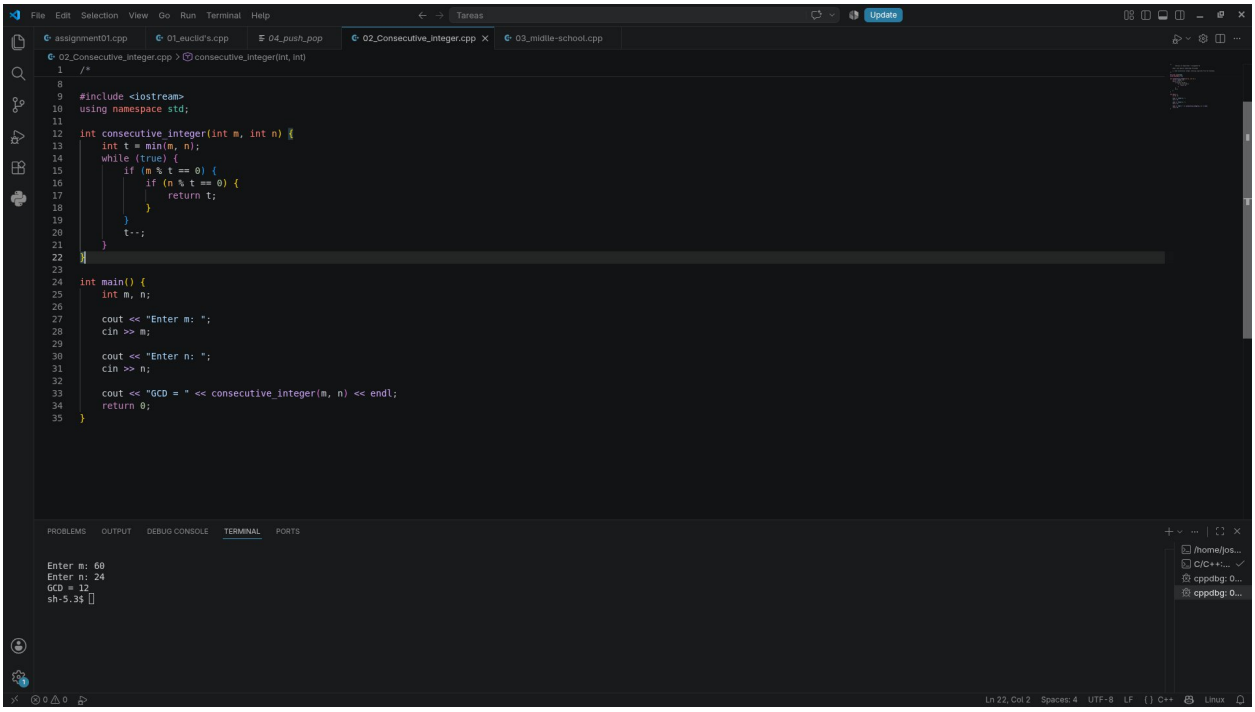
Name: Jose Ignacio Cabascango Fernandez

Link GitHub: https://github.com/CodeWithJose1/Analysis-of-Algorithms/tree/main/Assignament-01/src

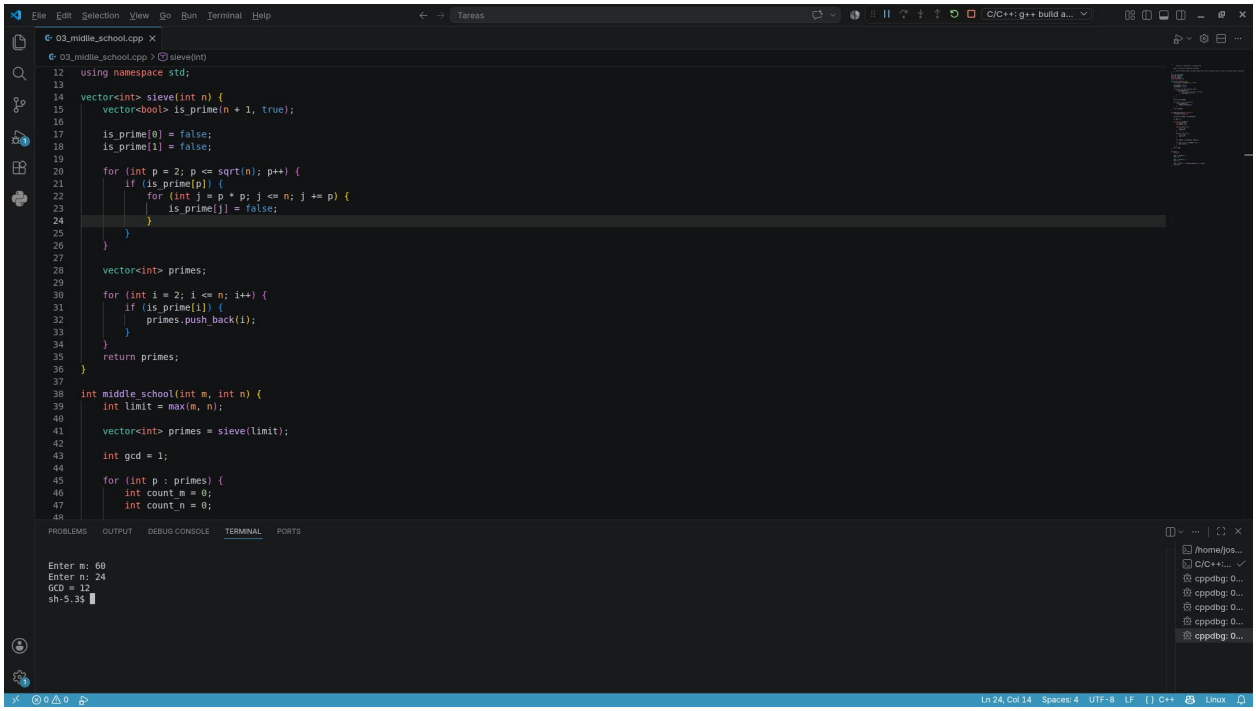
1. Code Euclid's Algorithms from the textbook.



2. Code Consecutive integer checking algorithm from the textbook.



3. Code the Middle-school procedure algorithm from the textbook (You will need to code the Sieve of Eratosthenes algorithm too)



4. Find the way C++ uses to push and pop elements to and from a list to behave like a stack. Create an example that showcases the idea.

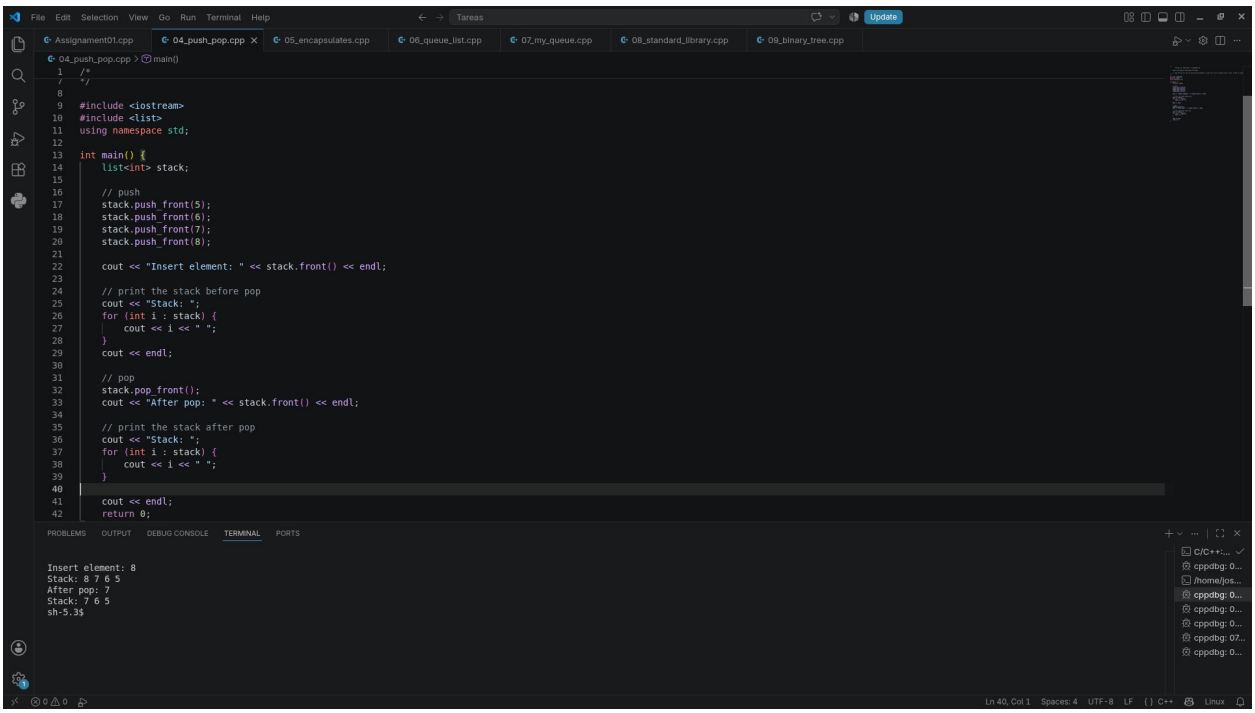
For the creation of a stack with the help of a list, we use std::list, which can be used to represent a stack. We must use the LIFO (Last in, First out) principle. To achieve this, we use the operations push_front() and pop_front().

- push_front() adds an element to the beginning of the list.
- pop_front() removes the first element from the list.
- front() allows you to check the first element without removing it.
- empty() allows checking if the list is empty.

For example, if the elements 5, 6, 7, and 8 are added using push_front(), the last element added, 8, remains at the top of the stack. When using pop_front(), the element 8 will be the first to exit.

Both push_front() and pop_front() have a time complexity of O(1) in std::list.

Example:



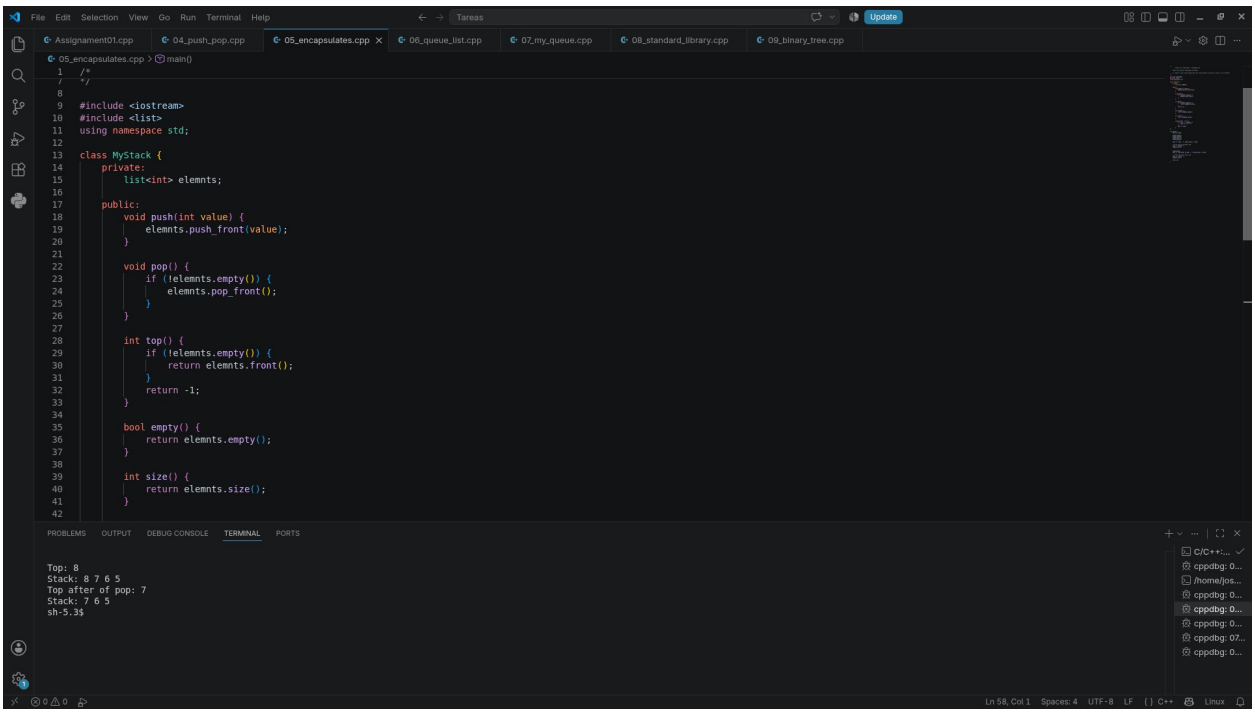
5. Create a class that encapsulates the stack behavior describe in point 4 (Ex: MyStack)

To encapsulate the behavior of a stack, a class called MyStack can be created. The class internally contains a std::list, while its operations are exposed through methods such as:

- push() to add elements.
- pop() to remove the top element.
- top() to query the top element.
- empty() to check if the stack is empty.
- size() to know the number of elements.

The objective of encapsulating the structure is to prevent external code from directly manipulating the list. Thus, the internal implementation remains protected and the user only needs to know the stack operations.

Example:



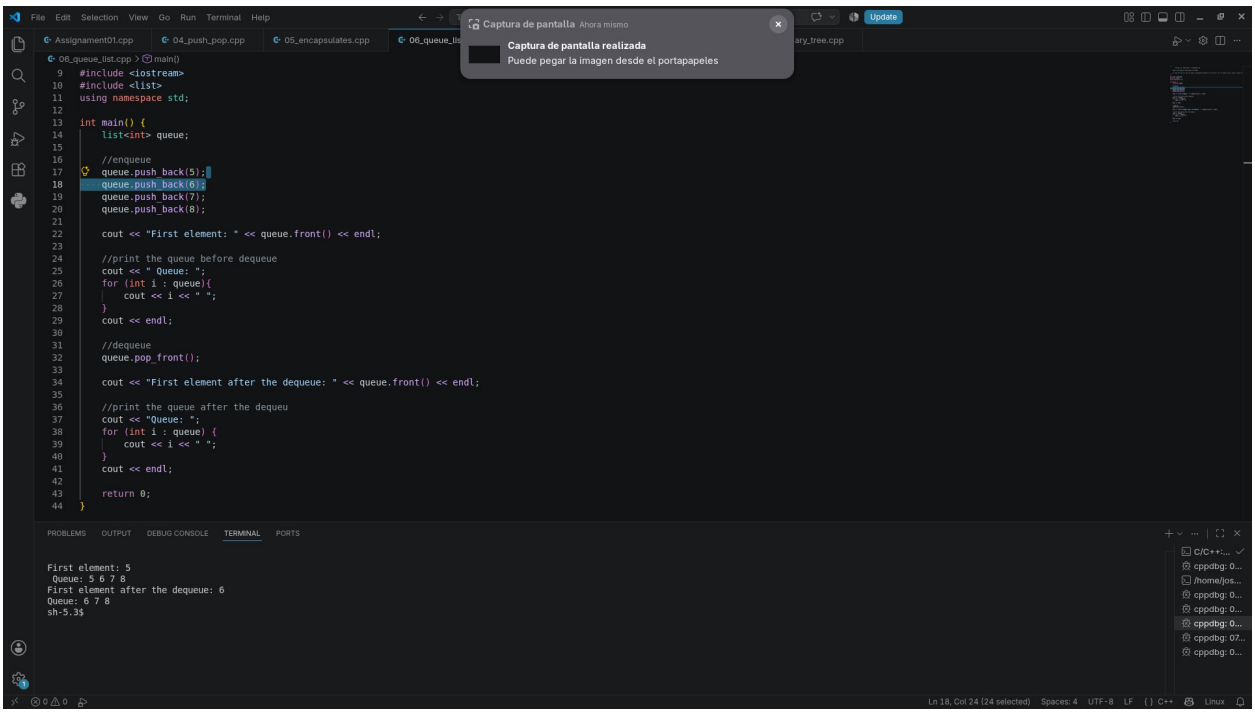
6. Find the way C++ uses to queue and dequeue elements to and from a list to behave like a queue. Create an example that showcases the idea.

For the creation of a queue with the help of a list, we use std::list, which can be used for the creation of a queue. We must use the FIFO principle (First In, First Out). To achieve this, we use the push_back() and pop_front() operations. The main operations are:

- push_back() adds an element to the end.
- pop_front() removes the first element.
- front() allows querying the first element
- back() allows you to query the last element.
- empty() checks if the queue is empty.

For example, if the elements 5, 6, 7, and 8 are added using push_back(), the last element added, 8, remains at the end of the queue. When using pop_front(), the element 5 will be the first to come out.

Example:



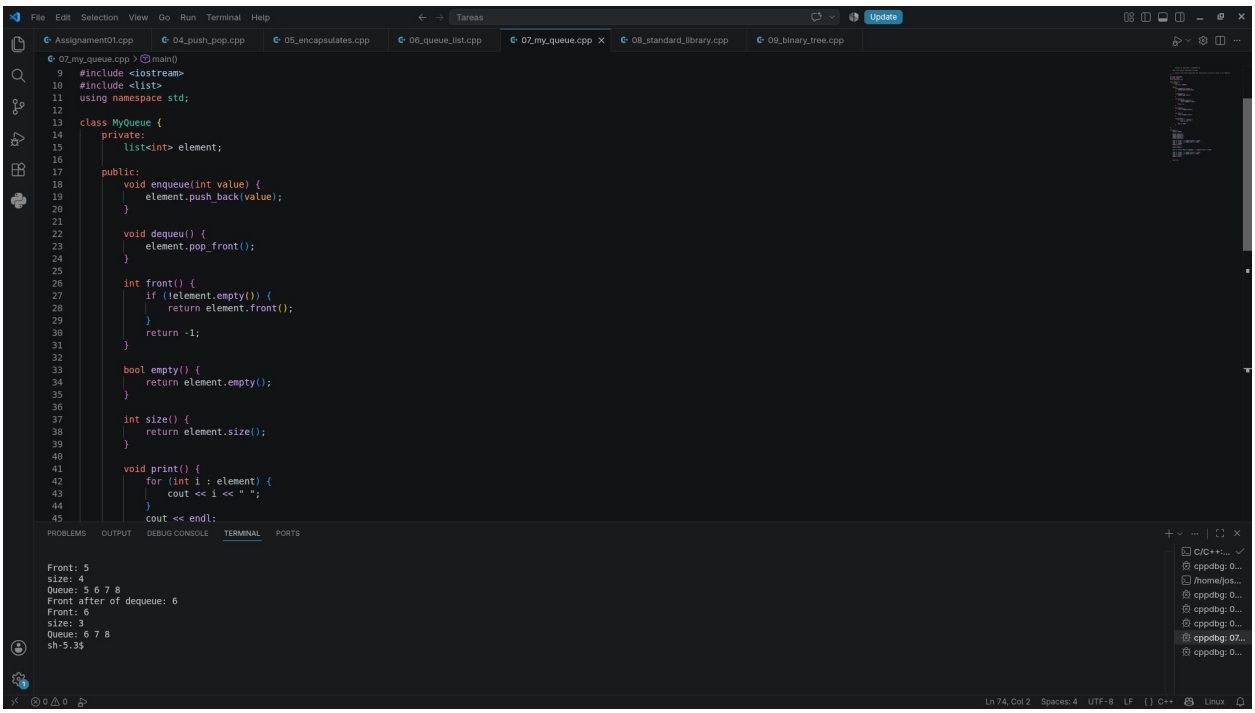
7. Create a class that encapsulates the stack behavior describe in point 6 (Ex: MyQueue)

The behavior of a queue can also be encapsulated using a custom class called MyQueue. This class internally uses a std::list and provides methods such as:

- enqueue() to add elements.
- dequeue() to remove the first element.
- front() to check the first element
- empty() to check if it is empty.
- size() to get the number of elements.

The advantage of creating MyQueue is that the internal implementation is separated from the way the structure is used. The main difference between a stack and a queue is the way elements are removed.

Example:



8. Explore which alternatives does C++ Standard Library provide to deal with stacks and queues. Provide examples.

The C++ Standard Library provides specialized structures for working with stacks and queues. These structures are known as container adaptors because they provide a specialized interface over an underlying container.

std::stack

std::stack represents a stack and uses the LIFO principle. Its main operations are:

- push() to insert.
- pop() to remove the top element.
- top() to query the top element.
- empty() to check if it is empty.
- size() to get the number of elements.

By default, std::stack uses std::deque as its internal container, although it can also use other compatible containers, such as std::vector or std::list.

std::queue

std::queue represents a queue and uses the FIFO principle. Its main operations are:

- push() to add an element to the end.
- pop() to remove the element from the front.
- front() to query the first element.
- back() to check the last element.
- empty() to check if it is empty.
- size() to get the number of elements.

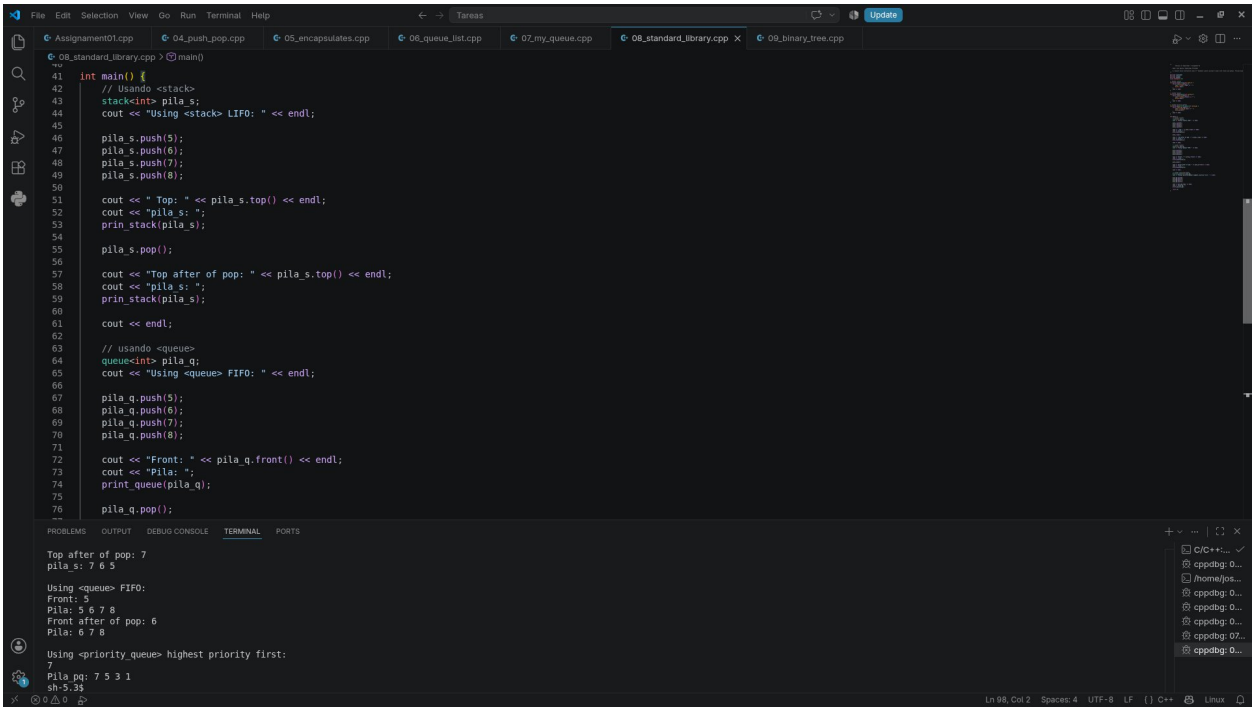
By default, std::queue uses std::deque as the underlying container, although it can also use std::list.

std::priority_queue

A priority_queue organizes its elemenTimes New Romants according to their priority. In this case, the element with the highest value has the highest priority and is available through top().

For example, if the values 10, 50, and 20 are added, the element available at the top will be 50. This allows the structure to be used in situations where the elements should not be processed simply according to the order in which they were inserted.

Example:

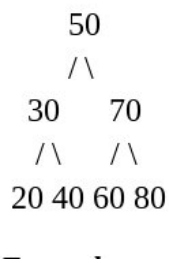


9. Create your own Binary Tree data structure and test the insertion of elements. Insertion will need its own algorithm, research on how to implement it.

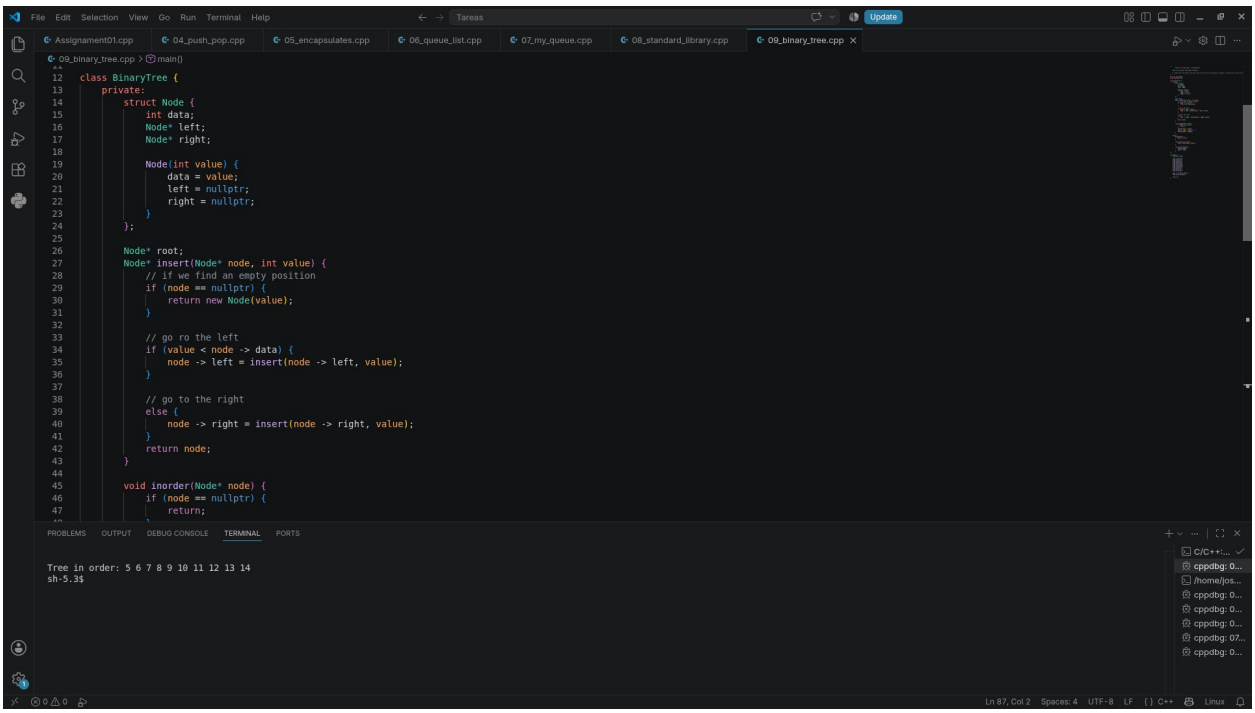
A Binary Tree is a data structure formed by nodes. Each node can have at most two children: a left child and a right child. A particular case is the Binary Search Tree (BST). In this type of tree, a rule is maintained that facilitates the search and insertion of elements. For a given node:

- Smaller values are placed in the left subtree.
- Values greater than the node are placed in the right subtree.
- Each of the subtrees must follow the same rule.

For example, if the values 50, 30, 70, 20, 40, 60, 80 are inserted, the resulting tree will be:



Example:


File Edit Selection View Go Run Terminal Help
Tareas
Update
Assignament01.cpp
04_push_pop.cpp
05_encapsulates.cpp
06_queue_list.cpp
07_my_queue.cpp
08_standard_library.cpp
09_binary_tree.cpp
09_binary_tree.cpp > main()
class BinaryTree {
private:
struct Node {
int data;
Node* left;
Node* right;
Node(int value) {
data = value;
left = nullptr;
right = nullptr;
}
};
Node* root;
Node* insert(Node* node, int value) {
// if we find an empty position
if (node == nullptr) {
return new Node(value);
}
// go ro the left
if (value < node -> data) {
node -> left = insert(node -> left, value);
}
// go to the right
else {
node -> right = insert(node -> right, value);
}
return node;
}
void inorder(Node* node) {
if (node == nullptr) {
return;
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Tree in order: 5 6 7 8 9 10 11 12 13 14
sh-5.3$
C/C++:...
cppdbg: 0...
/home/jos...
cppdbg: 0...
cppdbg: 0...
cppdbg: 0...
cppdbg: 07...
cppdbg: 0...
Ln 87, Col 2
Spaces: 4
UTF-8
LF
{} C++
Linux